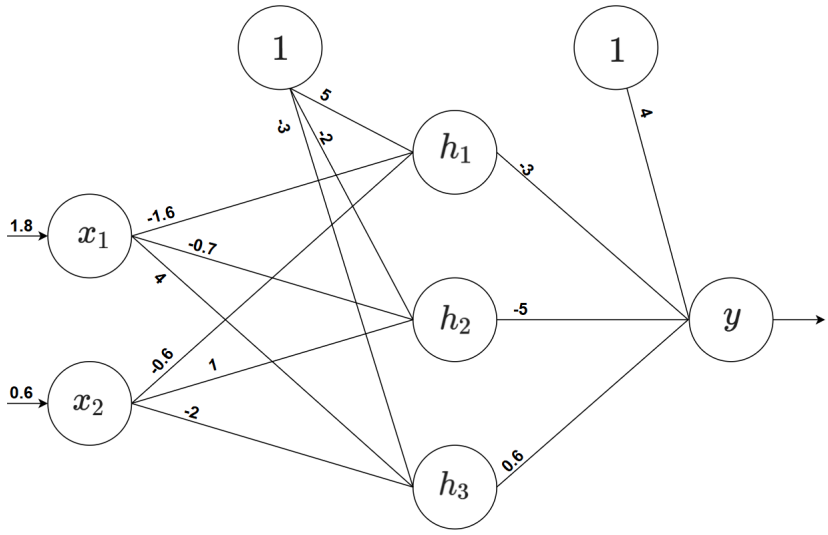
Universidade Federal do Pará
Instituto de Tecnologia (ITEC)
Faculdade de Engenharia da Computação e Telecomunicações (FCT)

Inteligência Computacional: Resolução da 2ª Avaliação

1) A rede neural abaixo é usada para classificação de risco de crédito, com as entradas representando a renda do cliente (x_1), medida em milhares de reais, e o histórico de endividamento (x_2). Considere que os possíveis resultados são "crédito rejeitado" e "crédito aprovado", mapeados em 0 e 1, respectivamente. A rede utiliza funções de ativação ReLU na camada escondida e função sigmoide $\left(\sigma(x) = \frac{1}{1+e^{-x}}\right)$ na saída.



Considerando os valores apresentados na imagem, e dado que o limiar de decisão da saída para esse problema é 0.7, calcule o valor da saída y da rede neural. O cliente terá o crédito aprovado?

Resolução:

Uma camada qualquer p de uma rede neural pode ser descrita em formato matricial como:

$$\mathbf{h}_p = \Psi(\mathbf{W_p^T}\ \mathbf{h_{p-1}} + \mathbf{b_p})$$

Sendo $\mathbf{W_p}$ e $\mathbf{b_p}$ os pesos e *biases* correspondentes a camada p e $\Psi(.)$ uma ativação qualquer. Pode-se considerar também $\mathbf{h_0} = \mathbf{x}$. Assim, para esse problema, tem-se que a camada escondida $\mathbf{h_1}$ é dada por:

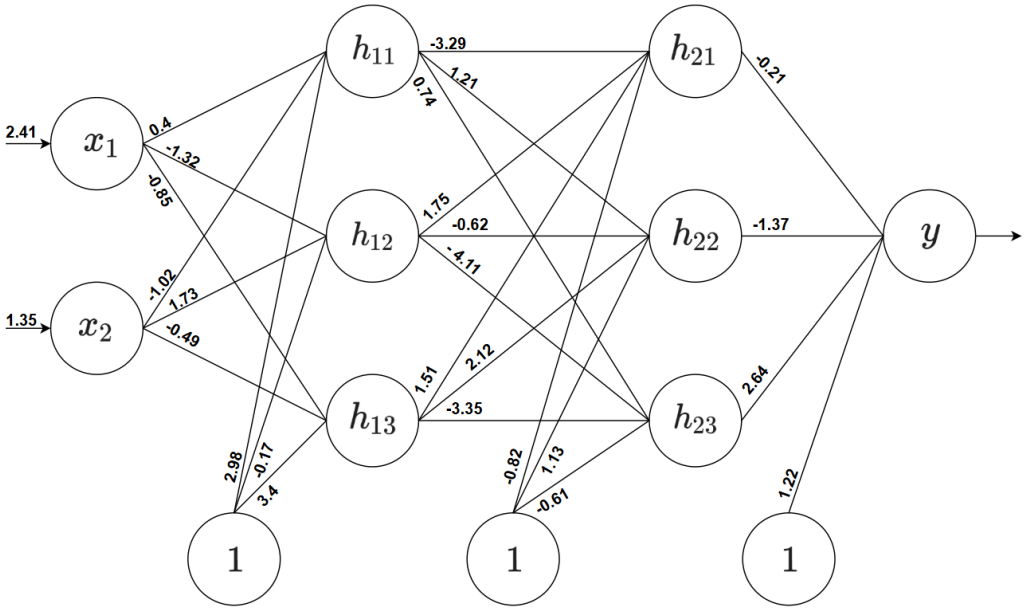
$$\mathbf{h_1} = \text{ReLU}\left(\begin{bmatrix} -1.6 & -0.6 \\ -0.7 & 1 \\ 4 & -2 \end{bmatrix} \begin{bmatrix} 1.8 \\ 0.6 \end{bmatrix} + \begin{bmatrix} 5 \\ -2 \\ -3 \end{bmatrix}\right) = \text{ReLU}\left(\begin{bmatrix} 1.76 \\ -2.66 \\ 3 \end{bmatrix}\right) = \begin{bmatrix} 1.76 \\ 0 \\ 3 \end{bmatrix}$$

Repetindo o processo para a saída, considerando $\sigma(.)$ como a ativação sigmoide, tem-se:

$$y = \sigma\left(\begin{bmatrix} -3 & -5 & 0.6 \end{bmatrix} \begin{bmatrix} 1.76 \\ 0 \\ 3 \end{bmatrix} + 4\right) = \sigma(0.52) = \frac{1}{1+e^{-0.52}} = 0.6271$$

Logo, como o limiar de decisão é 0.7 (ou seja a classe é dada por $I(y \leq 0.7)$, sendo $I(.)$ a função indicador), a saída está abaixo do limiar e a resposta é 0. Portanto **o cliente não terá o crédito aprovado**.

2) Considere uma rede neural destinada a prever a espécie de um conjunto de plantas a partir do comprimento e largura de suas pétalas. Essa rede possui a seguinte topologia:



Nessa rede neural, a ativação da primeira camada escondida é ReLU, da segunda é tangente hiperbólica e da camada de saída é sigmoide. Considere a entrada o vetor $\mathbf{x}$, a primeira camada escondida $\mathbf{h_1}$, a segunda camada escondida $\mathbf{h_2}$, o primeiro conjunto de *biases* $\mathbf{b_1}$, o segundo $\mathbf{b_2}$, o *bias* da saída b_3 e a saída y. Considere ainda $\sigma(\mathbf{x})$, $\Phi(\mathbf{x})$ e $\Psi(\mathbf{x})$ como as funções de ativação sigmoide, tangente hiperbólica e ReLU, respectivamente. Levando em conta os valores da imagem, **indique a notação matricial** (não é necessário encontrar o resultado) de **a)** $\mathbf{h_1}$ em função de $\mathbf{x}$ e $\mathbf{b_1}$, **b)** $\mathbf{h_2}$ em função de $\mathbf{h_1}$ e $\mathbf{b_2}$, **c)** y em função de $\mathbf{h_2}$ e b_3. Considere que a rede, durante o seu treinamento, passe por uma camada `Dropout(p=0.333)`, pergunta-se: **d)** quantos neurônios seriam descartados a cada iteração do cálculo do gradiente?

Resolução:

a) Uma camada qualquer p de uma rede neural pode ser descrita em formato matricial como:

$$\mathbf{h}_p = \Psi(\mathbf{W_p^T}\ \mathbf{h_{p-1}} + \mathbf{b_p})$$

Sendo $\mathbf{W_p}$ e $\mathbf{b_p}$ os pesos e *biases* correspondentes a camada p e $\Psi(.)$ uma ativação qualquer. Para o cálculo da primeira camada escondida, pode-se considerar também $\mathbf{h_0} = \mathbf{x}$. E, conforme indicado no enunciado, a primeira ativação é ReLU, indicada por $\Psi(.)$. Assim, tem-se que a primeira camada escondida $\mathbf{h_1}$ é dada por:

$$\mathbf{h_1} = \Psi(\mathbf{W_1^T}\ \mathbf{x} + \mathbf{b_1})$$

$$\mathbf{h_1} = \Psi\left(\begin{bmatrix} 0.4 & -1.02 \\ -1.32 & 1.73 \\ -0.85 & -0.49 \end{bmatrix} \begin{bmatrix} 2.41 \\ 1.35 \end{bmatrix} + \begin{bmatrix} 2.98 \\ -0.17 \\ 3.4 \end{bmatrix}\right)$$

b) Repetindo a mesma formulação matemática e considerando que a segunda ativação é tangente hiperbólica, representada por $\Phi(.)$, a segunda camada escondida é dada por:

$$\mathbf{h_2} = \Phi(\mathbf{W_2^T}\ \mathbf{h_1} + \mathbf{b_2})$$

$$\mathbf{h_2} = \Phi\left(\begin{bmatrix} -3.29 & 1.75 & 1.51 \\ 1.21 & -0.62 & 2.12 \\ 0.74 & -4.11 & -3.35 \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \end{bmatrix} + \begin{bmatrix} -0.82 \\ 1.13 \\ -0.61 \end{bmatrix}\right)$$

c) Repetindo a mesma formulação matemática e considerando que a segunda ativação é sigmoide, representada por $\sigma(.)$, a segunda camada escondida $\mathbf{h_2}$ é dada por:

$$y = \sigma(\mathbf{W_3^T}\ \mathbf{h_2} + b_3)$$

$$y = \sigma\left(\begin{bmatrix}-0.21 & -1.37 & 2.64\end{bmatrix}\begin{bmatrix}h_{21}\\ h_{22}\\ h_{23}\end{bmatrix} + 1.22\right)$$

d) O `Dropout` representa um descarte aleatório de neurônios dada a probabilidade de entrada. Na rede neural em questão, tem-se 3 neurônios tanto na primeira quanto na segunda camada escondida, e a probabilidade é $p = 0.333$. Logo, considerando a camada `Dropout` após a primeira camada ou após a segunda, o resultado é o mesmo: 3 neurônios $\times 0.333 = 1$ neurônio descartado, independente da camada avaliada.

3) Uma rede neural com apenas três neurônios é treinada para fazer regressão e prever o salário mínimo do trabalhador brasileiro. Considere um cenário com um conjunto de dados muito reduzido, o qual conta com apenas 4 exemplos. Para tornar este conjunto de treino mais próximo de i.i.d., usou-se o embaralhamento dele (*shuffle*). Os valores iniciais para o vetor de pesos são $\mathbf{w} = [0.1, -0.2,\ 0.3]$. Adotou-se uma taxa de aprendizado de $\eta = 0.2$. Dada a *loss function* MSE, calculou-se o vetor gradiente para cada um dos exemplos como: $\mathbf{g_1} = [1.5, -3.5, 2]$, $\mathbf{g_2} = [2,\ 0.5, -1]$, $\mathbf{g_3} = [-2.5,\ 3, -1.5]$, $\mathbf{g_4} = [4,\ 3, -1]$, onde o índice i de g_i já indica a sequência dos exemplos após o embaralhamento. Pergunta-se: **a)** Quais os valores finais, após uma época, para o vetor $\mathbf{w}$ considerando o tamanho B do *mini-batch*: como $B = 1$ exemplo? **b)** Considerando $B = 1$ e um treinamento de 50 épocas, qual seria o número de iterações do algoritmo SGD (número de atualizações dos pesos)? **c)** No caso de $B = 2$, qual o valor final dos pesos após uma época? **d)** Ainda considerando $B = 2$ e um treinamento de 100 épocas, qual o número de atualizações dos pesos?

Resolução:

a) Nesse primeiro caso, como $B = 1$, tem-se uma atulização dos pesos para cada valor de gradiente. Assim, **para a primeira atualização**:

$$\mathbf{w} = \mathbf{w} - \eta\mathbf{g_1}$$
$$\mathbf{w} = [0.1, -0.2,\ 0.3] - 0.2 \times [1.5, -3.5, 2]$$
$$\mathbf{w} = [0.1, -0.2,\ 0.3] - [0.3, -0.7, 0.4] = [-0.2, 0.5, -0.1]$$

Repetindo o procedimento, mas agora com os pesos atualizados, tem-se **para a segunda atualização**:

$$\mathbf{w} = \mathbf{w} - \eta\mathbf{g_2}$$
$$\mathbf{w} = [-0.2, 0.5, -0.1] - 0.2 \times [2,\ 0.5, -1]$$
$$\mathbf{w} = [-0.2, 0.5, -0.1] - [0.4, 0.1, -0.2] = [-0.6, 0.4, 0.1]$$

Repetindo o procedimento, mas agora com os pesos atualizados, tem-se **para a terceira atualização**:

$$\mathbf{w} = \mathbf{w} - \eta\mathbf{g_3}$$
$$\mathbf{w} = [-0.6, 0.4, 0.1] - 0.2 \times [-2.5,\ 3, -1.5]$$
$$\mathbf{w} = [-0.6, 0.4, 0.1] - [-0.5, 0.6, -0.3] = [-0.1, -0.2, 0.4]$$

Por fim, tem-se **para a quarta atualização**:

$$\mathbf{w} = \mathbf{w} - \eta\mathbf{g_4}$$
$$\mathbf{w} = [-0.1, -0.2, 0.4] - 0.2 \times [4,\ 3, -1]$$
$$\mathbf{w} = [-0.1, -0.2, 0.4] - [0.8, 0.6, -0.2] = [-0.9, -0.8, 0.6]$$

b) Conforme indicado no item anterior, quando $B = 1$, a cada época tem-se um total de 4 atu-

alizações de pesos. Assim, em 50 épocas, tem-se um total de 4 atualizações/época $\times$ 50 épocas $=$ 200 atualizações.

c) Como $B = 2$, tem-se um total de $\frac{\text{4 exemplos}}{\text{2 exemplos /mini-batch}} = 2$ minibatchs. Nesse caso, deve-se calcular um gradiente equivalente $\mathbf{g_e}$ para todos os 2 *mini-batchs*. Esse vetor é dado pela média dos gradientes do *mini-batch*. Assim, para o primeiro *mini-batch*, tem-se:

$$\mathbf{g_e} = \frac{\mathbf{g_1} + \mathbf{g_2}}{2} = 0.5 \times ([1.5, -3.5, 2] + [2,\ 0.5, -1])$$

$$\mathbf{g_e} = 0.5 \times ([3.5, -3, 1]) = [1.75, -1.5, 0.5]$$

Definido o gradiente, faz-se a **primeira atualização de pesos**:

$$\mathbf{w} = \mathbf{w} - \eta \mathbf{g_e}$$

$$\mathbf{w} = [0.1, -0.2,\ 0.3] - 0.2 \times [1.75, -1.5, 0.5]$$

$$\mathbf{w} = [0.1, -0.2,\ 0.3] - [0.35, -0.3, 0.1] = [-0.25, 0.1, 0.2]$$

Para o segundo *mini-batch*, primeiro deve-se calcular o gradiente equivalente. Assim, tem-se:

$$\mathbf{g_e} = \frac{\mathbf{g_3} + \mathbf{g_4}}{2} = 0.5 \times ([-2.5,\ 3, -1.5] + [4,\ 3, -1])$$

$$\mathbf{g_e} = 0.5 \times ([1.5,\ 6, -2.5]) = [0.75,\ 3, -1.25]$$

Definido o gradiente, e com os pesos atualizados da primeira iteração, faz-se a **segunda atualização de pesos**:

$$\mathbf{w} = \mathbf{w} - \eta \mathbf{g_e}$$

$$\mathbf{w} = [-0.25, 0.1, 0.2] - 0.2 \times [0.75,\ 3, -1.25]$$

$$\mathbf{w} = [-0.25, 0.1, 0.2] - [0.15, 0.6, -0.25] = [-0.4, -0.5,\ 0.45]$$

d) Conforme indicado no item anterior, quando $B = 2$, a cada época tem-se um total de 2 atualizações de pesos. Assim, em 100 épocas, tem-se um total de 2 atualizações/época $\times$ 100 épocas $=$ 200 atualizações.

4) Dada a definição utilizada no Keras da rede neural a seguir:

```
model = models.sequential
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(128, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))
model.add(Dropout(0.4))
model.add(Flatten())
model.add(Dense(units=128, activation='relu'))
model.add(Dense(units=10, activation='relu'))
model.summary()
```

E a saída da linha `model.summary()`:

Tabela 1: Descrição em termos de dimensão do modelo construído.

Layer (type)	**Output shape**	**Param #**
conv2d_1 (Conv2D)	(None, 26, 26, 32)	$320 = ((3 \times 3) \times 1 + 1) \times 32)$
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_2 (Conv2D)	(None, 11, 11, 128)	$36\,992 = ((3 \times 3) \times 32 + 1) \times 128)$
max_pooling2d_2 (MaxPooling2D)	(None, 5, 5, 128)	0
droupout_1 (Dropout)	(None, 5, 5, 128)	0
flatten_1 (Flatten)	(None, 3200)	0
dense_1 (Dense)	(None, 128)	$409\,728 = (3200 + 1) \times 128$
dense_2 (Dense)	(None, 10)	$1\,290 = (128 + 1) \times 10$

Pergunta-se: **a)** qual é a quantidade de parâmetros treináveis (`Param #`) de cada camada acima? **(b)** qual é o total de parâmetros treináveis de toda a rede?

Resolução:

a) A quantidade de parâmetros treináveis, nesta questão, é aplicável apenas às camadas convolucionais (`conv2d`) e densas (`dense`). Nesse sentido, o número de parâmetros treináveis para uma camada convolucional é dado por:

$$\text{conv2_param} = [(x_{\text{kernel}} \times y_{\text{kernel}} \times \text{depth}) + 1] \times \#\text{filtros}.$$

Para camadas densas, tem-se que:

$$\text{dense_param} = (\text{tamanho da entrada} + 1) \times \#\text{neurônios}$$

Portanto, na Tabela 1 tem-se os resultados para cada camada, usando as fórmulas acima.

b) Somando os parâmetros de cada camada, tem-se que a quantidade total de parâmetros treináveis é: **448 330**.

5) Qual a saída de uma camada `MaxPooling2D` (por exemplo, a do `Keras`) criada com a instrução `layers.MaxPooling2D((5, 5), padding='same', strides=(2, 2))`, quando sua entrada é um tensor de dimensão 4×4 dado por

$$\mathbf{X} = \begin{bmatrix} 1 & 110 & 3 & 10 \\ 170 & 2 & 34 & -22 \\ 3 & 1 & -255 & 220 \\ 1 & -23 & 45 & 27 \end{bmatrix}.$$

Resolução:

Para resolver essa questão, é necessário saber que, independente do valor do `stride`, atributo `padding='same'` irá adicionar $F - 1$ linhas e colunas, em que F é a dimensão do kernel da camada de `MaxPooling2D`, $F = 5$. Portanto, após fazer o padding, temos o seguinte tensor:

$$\mathbf{Z}_{\text{padded}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 110 & 3 & 10 & 0 & 0 \\ 0 & 0 & 170 & 2 & 34 & -22 & 0 & 0 \\ 0 & 0 & 3 & 1 & -255 & 220 & 0 & 0 \\ 0 & 0 & 1 & -23 & 45 & 27 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Considerando um `stride` 2×2, temos, portanto, a saída final sendo um tensor com dimensões 2×2:

$$\mathbf{Z}_{\text{out}} = \begin{bmatrix} 170 & 220 \\ 170 & 220 \end{bmatrix}$$

6) O tensor de entrada $\mathbf{X}$ da camada convolucional de número $c = 5$ de uma rede neural tem dimensão $4 \times 4 \times 2$, sendo dado por:

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 4 & 3 & 2 \\ 0 & 2 & 5 & 4 \\ 3 & 0 & 8 & 0 \\ 1 & 5 & 0 & 3 \end{bmatrix} \quad \text{e} \quad \mathbf{X}_2 = \begin{bmatrix} -1 & 0 & 0 & -2 \\ 0 & 2 & 0 & -4 \\ -3 & 0 & 8 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}.$$

Essa camada usa um só filtro com os kernels dados por

$$\mathbf{F}_{5,1,1} = \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix},$$

$$\mathbf{F}_{5,1,2} = \begin{bmatrix} -1 & 1 \\ 0 & 1 \end{bmatrix}.$$

A nomenclatura do filtro $\mathbf{F}_{c,f,d}$ denota a camada c, filtro f e d a dimensão da profundidade. A convolução é feita com `stride` $= (1,\ 1)$ e com `padding` (`padding`='same' na convenção do `Keras`), de forma que a saída $\mathbf{Y}$ de cada convolução tenha a mesma dimensão da entrada $\mathbf{X}$. Qual a dimensão e valores do tensor de saída desta primeira camada escondida, considerando-se que a função de ativação é a ReLU e o bias é $b = 2$?

Resolução:
A solução para esta questão possui variações de resultados finais a depender de qual coluna/matriz o zero *padding* é realizado, totalizando três possíveis configurações. Ademais, o "bias" é aplicado à matriz obtida pela soma das convoluções com todos os kernels. Logo, considerando a configuração do primeiro *padding* como sendo na última coluna à direita e na última linha, temos:
Para o 1º filtro:

$$\mathbf{Z}_5 = \mathbf{X}_{1_{padded}} * \mathbf{F}_1 + \mathbf{X}_{2_{padded}} * \mathbf{F}_2$$

Portanto, realizando as convoluções, temos que:

$$\mathbf{Z}_5 = \begin{bmatrix} 1 & 4 & 3 & 2 & 0 \\ 0 & 2 & 5 & 4 & 0 \\ 3 & 0 & 8 & 0 & 0 \\ 1 & 5 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix} + \begin{bmatrix} -1 & 0 & 0 & -2 & 0 \\ 0 & 2 & 0 & -4 & 0 \\ -3 & 0 & 8 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} -1 & 1 \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{Z}_5 = \begin{bmatrix} 3 & 6 & 11 & 10 \\ 8 & -1 & 25 & 4 \\ 0 & 18 & 5 & 6 \\ 6 & 5 & 3 & 3 \end{bmatrix} + \begin{bmatrix} 3 & 0 & -6 & 2 \\ 2 & 6 & -4 & 4 \\ 3 & 8 & -8 & 0 \\ -1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 6 & 6 & 5 & 12 \\ 10 & 5 & 21 & 8 \\ 3 & 26 & -3 & 6 \\ 5 & 5 & 3 & 3 \end{bmatrix}$$

Somando a $\mathbf{Z}_5$ o "bias" 1 multiplicado pelo peso 2, temos que a saída final é:

$$\mathbf{Y}_5 = \text{ReLU}\left\{ \mathbf{Z}_5 + \begin{bmatrix} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{bmatrix} \right\} = \text{ReLU}\left\{ \begin{bmatrix} 8 & 8 & 7 & 14 \\ 12 & 7 & 23 & 10 \\ 5 & 28 & -1 & 8 \\ 7 & 7 & 5 & 5 \end{bmatrix} \right\} = \begin{bmatrix} 8 & 8 & 7 & 14 \\ 12 & 7 & 23 & 10 \\ 5 & 28 & 0 & 8 \\ 7 & 7 & 5 & 5 \end{bmatrix}$$

Alternativamente, uma possível e válida solução seria considerar um zero *padding* na primeira linha e coluna. Desse modo, teríamos que:

$$\mathbf{Z}_5 = \begin{bmatrix} 0 & 1 & 4 & 3 & 2 \\ 0 & 0 & 2 & 5 & 4 \\ 0 & 3 & 0 & 8 & 0 \\ 0 & 1 & 5 & 0 & 3 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix} + \begin{bmatrix} 0 & -1 & 0 & 0 & -2 \\ 0 & 0 & 2 & 0 & -4 \\ 0 & -3 & 0 & 8 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} -1 & 1 \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{Z}_5 = \begin{bmatrix} 1 & 3 & 6 & 11 \\ -3 & 8 & -1 & 25 \\ 2 & 0 & 18 & 5 \\ 1 & 6 & 5 & 3 \end{bmatrix} + \begin{bmatrix} -1 & 3 & 0 & -6 \\ -3 & 2 & 6 & -4 \\ -2 & 3 & 8 & -8 \\ 1 & -1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 6 & 6 & 5 \\ -6 & 10 & 5 & 21 \\ 0 & 3 & 26 & -3 \\ 2 & 5 & 5 & 3 \end{bmatrix}$$

Somando a $\mathbf{Z}_5$ o "bias" 1 multiplicado pelo peso 2, temos que a saída final é:

$$\mathbf{Y}_5 = \text{ReLU}\left\{ \mathbf{Z}_5 + \begin{bmatrix} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{bmatrix} \right\} = \text{ReLU}\left\{ \begin{bmatrix} 2 & 8 & 8 & 7 \\ -4 & 12 & 7 & 23 \\ 2 & 5 & 28 & -1 \\ 4 & 7 & 7 & 5 \end{bmatrix} \right\} = \begin{bmatrix} 2 & 8 & 8 & 7 \\ 0 & 12 & 7 & 23 \\ 2 & 5 & 28 & 0 \\ 4 & 7 & 7 & 5 \end{bmatrix}$$

Por fim, poderíamos ter um zero *padding* na primeira linha das entradas e na primeira coluna, como seguem os cálculos a seguir.

$$\mathbf{Z}_5 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 4 & 3 & 2 \\ 0 & 0 & 2 & 5 & 4 \\ 0 & 3 & 0 & 8 & 0 \\ 0 & 1 & 5 & 0 & 3 \end{bmatrix} * \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & -2 \\ 0 & 0 & 2 & 0 & -4 \\ 0 & -3 & 0 & 8 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} -1 & 1 \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{Z}_5 = \begin{bmatrix} -1 & -2 & 5 & 4 \\ 1 & 3 & 6 & 11 \\ -3 & 8 & -1 & 25 \\ 2 & 0 & 18 & 5 \end{bmatrix} + \begin{bmatrix} -1 & 0 & 0 & -2 \\ -1 & 3 & 0 & -6 \\ -3 & 2 & 6 & -4 \\ -2 & 3 & 8 & -8 \end{bmatrix} = \begin{bmatrix} -2 & -2 & 5 & 2 \\ 0 & 6 & 6 & 5 \\ -6 & 10 & 5 & 21 \\ 0 & 3 & 26 & -3 \end{bmatrix}$$

Somando a $\mathbf{Z}_5$ o "bias" 1 multiplicado pelo peso 2, temos que a saída final é:

$$\mathbf{Y}_5 = \text{ReLU}\left\{ \mathbf{Z}_5 + \begin{bmatrix} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{bmatrix} \right\} = \text{ReLU}\left\{ \begin{bmatrix} 0 & 0 & 7 & 4 \\ 2 & 8 & 8 & 7 \\ -4 & 12 & 7 & 23 \\ 2 & 5 & 28 & -1 \end{bmatrix} \right\} = \begin{bmatrix} 0 & 0 & 7 & 4 \\ 2 & 8 & 8 & 7 \\ 0 & 12 & 7 & 23 \\ 2 & 5 & 28 & 0 \end{bmatrix}$$

7) Uma implementação da rede YOLO para detecção de objetos busca detectar $C = 4$ classes, onde 0, 1, 2 e 3 representam cavalo, cachorro, gato e pássaro. Esta YOLO adota $B = 2$ *bounding boxes* por bloco do grid, e organiza, para cada bloco do grid, as 14 informações correspondentes de acordo com: ρ_1, x_1, y_1, h_1, w_1, ρ_2, x_2, y_2, h_2, w_2, c_1, c_2, c_3, c_4. onde ρ_1 e ρ_2 são as "confianças" ("confidence"), para cada *bounding box*. Para uma dada imagem de entrada com 224×224 pixels, as saídas desta YOLO foram:

```
0.85, 0.2, 0.1, 0.2, 0.4, 0.4, 0.25, 0.15, 0.2, 0.3, 0.1, 0.2, 0.5, 0.2
0.3, 0.1, 0.8, 0.2, 0.3, 0.8, 0.2, 0.7, 0.2, 0.1, 0.6, 0.1, 0.1, 0.2
0.2, 0.7, 0.1, 0.2, 0.4, 0.7, 0.75, 0.3, 0.2, 0.2, 0.2, 0.6, 0.1, 0.1
0.6, 0.8, 0.7, 0.2, 0.1, 0, 0.6, 0.8, 0.1, 0.3, 0.1, 0.6, 0.1, 0.2
```

Em que a primeira linha acima se refere ao bloco A no canto superior esquerdo, a segunda ao bloco B no canto superior direito, a terceira ao bloco C no canto inferior esquerdo e, por fim, a do bloco D no canto inferior direito. Considere como **0.65** o valor do limiar de confiança para se declarar que há um objeto no *bounding box*. Pergunta-se:

a) Considerando o limiar de confiança adotado, qual o total de *bounding boxes* identificados pela rede, levando em conta os quatro blocos. **b)** Quais são as classes dos objetos detectados?

Resolução:

a) O primeiro passo para a resolução desta questão passa por identificar o limiar de confiança apontado pela rede YOLO para as quatro regiões mencionadas.

Para o bloco A: Apenas uma *bounding box*, com limiar de **0.85.**
Para o bloco B: Apenas uma *bounding box*, com limiar de **0.8.**
Para o bloco C: Apenas uma *bounding box*, com limiar de **0.7.**
Para o bloco D: Nenhuma *bounding box* detectada.
Portanto, temos um total de 3 *bounding box* detectados.

b) Assumindo os *bounding box* detectados, é preciso verificar a probabilidade associada a cada classe. Portanto:

Para o bloco A: O *bounding box* detectado refere-se à classe "gato".
Para o bloco B: O *bounding box* detectado refere-se à classe "cavalo".
Para o bloco C: O *bounding box* detectado refere-se à classe "cachorro".